

BAB 6

ALGORITMA KLASIFIKASI SUPPORT VECTOR MACHINE (SVM)

6.1 TUJUAN PERCOBAAN

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Mengetahui konsep dasar algoritma SVM
2. Mengimplementasikan algoritma SVM berbasis Stochastic Gradient Descent
3. Mengimplementasikan salah satu metode SVM untuk klasifikasi multi kelas

6.2 ALAT-ALAT YANG DIGUNAKAN

Pada percobaan ini, akan digunakan berbagai alat dan bahan yang mendukung kegiatan praktikum. Alat-alat tersebut meliputi:

- Daftar alat utama: OS Windows/Linux/macOS, Anaconda, Google Colab
- Pustaka: pandas, numpy, matplotlib, seaborn

6.3 DASAR TEORI

6.3.1 SVM UNTUK KLASIFIKASI BINER

Support Vector Machine (SVM) merupakan sebuah algoritma klasifikasi yang mampu mengenali dua kelas dalam data. Dengan demikian, SVM termasuk dalam algoritma klasifikasi biner, karena hanya mampu mengenali dua kelas, yaitu positif dan negatif. Cara kerja SVM adalah mencari hyperplane dengan fungsi $w \cdot x + b = 0$ yang berfungsi sebagai garis pemisah antar dua kelas. Kelas positif akan memiliki nilai $w \cdot x + b \geq 1$ sedangkan kelas negatif memiliki nilai $w \cdot x + b \leq -1$.

SVM mencari hyperplane optimal, yaitu hyperplane yang memiliki cost function minimal. Cost function digunakan untuk mengukur seberapa baik suatu hyperplane. Terdapat banyak cost function yang dapat digunakan pada SVM, salah satunya adalah:

$$J(w) = \frac{1}{2} \|w\|^2 + C \left[\frac{1}{N} \sum_i^n \max(0, 1 - y_i * (w \cdot x_i + b)) \right]$$

Keterangan:

- $J(w)$: Cost function dari hyperplane w
 C : Parameter regularization
 N : Banyak data
 y : label data
 x : data

Optimasi hyperplane memerlukan perhitungan gradien dari cost function. Persamaan gradien dari cost function yang digunakan adalah

$$\nabla_w J(w) = \frac{1}{N} \sum_i^n \begin{cases} w & \text{jika } \max(0, 1 - y_i * (w \cdot x_i + b)) = 0 \\ w - C y_i x_i & \text{lainnya} \end{cases}$$

Nilai gradien dari cost function digunakan oleh algoritma Stochastic Gradient Descent (SGD) untuk melakukan minimasi nilai cost function. Cara kerja algoritma SGD secara umum adalah:

1. Hitung nilai gradien dari cost function.
2. Lakukan perubahan bobot dengan arah yang berlawanan terhadap nilai gradien.
3. Ulangi langkah 1-2 sampai konvergen.

Tahapan pelatihan pada SVM dilakukan dengan cara optimasi cost function, salah satunya dengan SGD. Tahapan pengujian dilakukan dengan menghitung nilai *dot product* antara bobot dengan data uji. Selanjutnya, kelas data uji ditentukan berdasarkan tanda (positif atau negatif) dari nilai *dot product*.

6.3.2 SVM UNTUK KLASIFIKASI *MULTICLASS*

Selain didesain untuk melakukan proses klasifikasi biner, SVM dapat dimodifikasi untuk menyelesaikan klasifikasi multi kelas dengan cara mengubah permasalahan multi kelas menjadi beberapa permasalahan klasifikasi biner. Dua metode umum untuk menyelesaikan permasalahan SVM multi kelas adalah one-vs-rest dan one-vs-one.

Metode on-vs-rest (atau biasa juga disebut one-vs-all atau one-against-all) membentuk sebuah model SVM untuk setiap kelas yang ada. Sebagai contoh, pada *dataset* Iris yang terdiri dari 3 kelas, maka akan terbentuk 3 model SVM sebagai berikut:

1. SVM1, berfungsi mengenali kelas Iris-setosa. Model ini dilatih menggunakan *dataset* Iris, yang mana data latih dengan kelas Iris-setosa diberi target/label 1 sedangkan kelas lainnya diberi target/label -1.
2. SVM2, berfungsi mengenali kelas Iris-versicolor. Model ini dilatih menggunakan *dataset* Iris, yang mana data latih dengan kelas Iris-versicolor diberi target/label 1 sedangkan kelas lainnya diberi target/label -1.
3. SVM3, berfungsi mengenali kelas Iris-virginica. Model ini dilatih menggunakan *dataset* Iris, yang mana data latih dengan kelas Iris-virginica diberi target/label 1 sedangkan kelas lainnya diberi target/label -1.

Strategi kedua adalah one-vs-one yang memecah permasalahan multi kelas menjadi permasalahan klasifikasi biner. Metode one-vs-one membentuk $K(K-1)/2$ model, dengan K menyatakan banyaknya kelas pada data. Setiap model ditujukan untuk mengenali pasangan kelas tertentu. Dengan menggunakan contoh *dataset* Iris, maka akan terbentuk $3(3-1)/2=3$ model sebagai berikut:

1. SVM1 mengenali Iris-setosa dan Iris-versicolor
2. SVM2 mengenali Iris-setosa dan Iris-virginica
3. SVM3 mengenali Iris-versicolor dan Iris virginica.

Praktikum ini akan menggunakan metode one-vs-rest untuk melakukan proses klasifikasi data Iris. Metode one-vs-rest memanfaatkan klasifikasi SVM biner yang telah diimplementasikan pada praktikum sebelumnya.

6.4 PROSEDUR PERCOBAAN

6.4.1 IMPOR DATA

Unduh *dataset* yang akan digunakan pada praktikum kali ini. Anda dapat menggunakan aplikasi wget untuk mendownload *dataset* dan menyimpannya dalam Google Colab. Unduh *dataset* dari link berikut:

<https://gist.github.com/Thanatoz-1/9e7fdfb8189f0cdf5d73a494e4a6392a>

Setelah *dataset* berhasil diunduh, langkah berikutnya adalah membaca *dataset* dengan memanfaatkan fungsi `readcsv` dari library `pandas`. Lakukan pembacaan berkas csv ke dalam `DataFrame` dengan nama data menggunakan fungsi `readcsv`. Jangan lupa untuk melakukan import library `pandas` terlebih dahulu

1. Percobaan Import Data

```
import pandas as pd
import numpy as np
data = pd.read_csv('iris.csv')
```

Tampilkan beberapa baris dari *dataset* untuk mendapatkan informasi singkat mengenai isi data. Gunakan fungsi `head()` untuk menampilkan 5 data pertama.

```
data.head()
```

6.4.2 KLASIFIKASI BINER

Karena pada dasarnya metode Support Vector Machine (SVM) dasar hanya mampu melakukan klasifikasi data yang terdiri dari dua kelas, padahal *dataset* iris memiliki 3 kelas, maka untuk percobaan ini, salah satu kelasnya, yaitu Iris-virginica perlu dihapus. *Cell berikut* menghapus data yang memiliki kelas Iris-virginica

```
data.drop(data[data['target']=='Iris-virginica'].index,inplace=True)
```

SVM juga memiliki keterbatasan, yaitu hanya mampu menerima kelas dalam bentuk numerik. *Cell berikut* mengubah kelas menjadi bilangan, Iris setosa menjadi -1 dan Iris-versicolor menjadi 1

```
data['target']=data['target'].map({'Iris-setosa':-1,'Iris-versicolor':1})
```

Tampilkan beberapa data untuk mengecek hasilnya

```
data.head()
```

1. Percobaan Membagi Data

Metode pembelajaran mesin memerlukan dua jenis data, yaitu data latih dan data uji. Data latih digunakan untuk proses pelatihan (*training*) model klasifikasi, sementara data uji digunakan untuk proses evaluasi metode klasifikasi. Data uji dan data latih perlu dibuat terpisah (mutually exclusive) agar untuk melihat tingkat generalisasi data. Artinya, model klasifikasi dapat mengenali data-data yang belum pernah dilatih sebelumnya dengan baik. Data uji dan data latih dapat dibuat dengan cara membagi *dataset* dengan rasio tertentu, misalnya 80% data latih dan 20% data uji.

Library Scikit-learn memiliki fungsi `train_test_split` pada modul `model_selection` untuk membagi *dataset* menjadi data latih dan data uji. Bagilah *dataset* anda menjadi dua, yaitu **data_latih** dan **data_uji** dengan menggunakan kode berikut:

```
from sklearn.model_selection import train_test_split
data_latih,data_uji = train_test_split(data,test_size=0.2)
```

Parameter `test_size` pada fungsi `train_test_split` menunjukkan persentase data uji yang dihasilkan. Pada contoh ini, 20% data digunakan sebagai data uji.

Kemudian, tampilkan banyaknya data pada **data_latih** dan **data_uji**. Seharusnya **data_latih** terdiri dari 80 data, dan **data_uji** terdiri dari 20 data

```
print(data_uji.shape[0])
print(data_latih.shape[0])
```

Pisahkan label/kategori dari data latih dan data uji menjadi variabel tersendiri. Beri nama variabelnya **label_latih** dan **label_uji**. Fungsi `pop()` pada *DataFrame* berfungsi untuk mengambil suatu kolom dari

DataFrame, hampir sama dengan fungsi `pop()` pada struktur data *stack*.

```
label_latih = data_latih.pop('target')  
label_uji = data_uji.pop('target')
```

2. Percobaan Proses *Training*

Tujuan dari algoritma SVM adalah meminimalkan nilai *cost function*. Penghitungan nilai minimal dapat dilakukan dengan menghitung nilai gradien dari *cost function* terlebih dahulu. Fungsi di bawah ini berguna untuk menghitung nilai gradien *cost function* seperti yang tertera pada bagian teori.

```
1 def hitung_cost_gradient(W,X,Y,regularization):  
2     jarak = 1 - (Y* np.dot(X,W))  
3     dw = np.zeros(len(W))  
4     if max(0,jarak)==0:  
5         di=W  
6     else:  
7         di = W - (regularization * Y*X)  
8     dw += di  
9     return dw
```

Terdapat beberapa cara untuk meminimalkan nilai *cost function*, salah satunya menggunakan Stochastic Gradient Descent (SGD) untuk melakukan minimasi. Minimasi *cost function* merupakan inti dari algoritma SVM. Fungsi di bawah ini merupakan implementasi algoritma SGD

```
1 from sklearn.utils import shuffle  
2 def sgd(data_latih,label_latih,learning_rate =  
3     0.000001,max_epoch=1000,regularization=10000):  
4     data_latih = data_latih.to_numpy()  
5     label_latih = label_latih.to_numpy()  
6     bobot = np.zeros(data_latih.shape[1])  
7     for epoch in range(1,max_epoch):  
8         X,Y =shuffle(data_latih,label_latih,random_state=101)  
9         for index,x in enumerate(X):  
10             delta=hitung_cost_gradient(bobot,x,Y[index],regularization)  
11             bobot = bobot - (learning_rate * delta)  
12     return bobot
```

Pada algoritma SGD, dilakukan perulangan sebanyak **max_epoch** dan dilakukan perhitungan bobot, yang merupakan parameter *hyperplane*. Pada setiap perulangan dilakukan pengacakan data, agar tidak terjadi pola perhitungan bobot yang sama. Nilai bobot diperoleh dari bobot dikurangi gradien dikali dengan *learning rate*. Nilai *learning rate* merupakan sebuah parameter yang mengatur seberapa besar perubahan

```
W = sgd(data_latih,label_latih)  
print(W)
```

3. Percobaan Proses *Testing*

Proses *testing* dilakukan dengan menghitung nilai *dot product* antara bobot hasil *training* dengan data uji. Kelas data ditentukan berdasarkan tanda (positif atau negatif) dari hasil *dot product* tersebut. Fungsi berikut mengimplementasikan proses *testing*.

```
1 def testing(W,data_uji):  
2     prediksi = np.array([])  
3     for i in range(data_uji.shape[0]):  
4         y_prediksi = np.sign(np.dot(W,data_uji.to_numpy()[i]))  
5         prediksi = np.append(prediksi,y_prediksi)  
6     return prediksi
```

Lakukan pengujian menggunakan seluruh data uji. Bandingkan hasilnya dengan nilai label sebenarnya untuk menghitung berapa banyak data uji yang berhasil diprediksi dengan benar.

```
y_prediksi = testing(W,data_uji)
print(sum(y_prediksi==label_uji))
```

6.4.3 KLASIFIKASI MULTI-CLASS

Gunakan *dataset* yang sudah diimport pada subbab 6.4.1 (IMPOR DATA), dan kali ini jangan hapus *class* Iris-Virginica, lalu bagi *dataset* menjadi data *training* dan data uji, seperti pada percobaan 1. Membagi data pada subbab 6.4.2. KLASIFIKASI BINER.

1. Percobaan Pembuatan Data Latih One-vs-rest

Metode one-vs-rest memerlukan tiga jenis data latih yang diperlukan untuk melatih tiga SVM yang berbeda pada *dataset* Iris. Fungsi **buat_trainingset** digunakan untuk membentuk tiga *dataset* tersebut.

```
1 def buat_trainingset(dataset):
2     trainingset = {}
3     kolom_kelas = dataset.columns[-1]
4     list_kelas = dataset[kolom_kelas].unique()
5     for kelas in list_kelas:
6         data_temp = dataset.copy(deep=True)
7         data_temp[kolom_kelas]=data_temp[kolom_kelas].map({kelas:1})
8         data_temp[kolom_kelas]=data_temp[kolom_kelas].fillna(-1)
9         trainingset[kelas]=data_temp
10    return trainingset
11
```

Fungsi **buat_trainingset** menghasilkan *dictionary* dengan key berupa kelas-kelas yang terdapat pada data. Value pada *dictionary* adalah data latih dengan nilai kelas yang dimodifikasi. Kelas yang sama dengan key akan diberi nilai 1, selain itu diberi nilai -1. Gunakan fungsi **buat_trainingset** ini untuk membentuk data latih dengan nama variabel *trainingset* yang akan digunakan pada proses *training*.

```
trainingset = buat_trainingset(data_latih)
```

Tampilkan isi *trainingset* agar Anda dapat memahami struktur dari variabel tersebut.

```
print(trainingset)
```

2. Percobaan Proses Training

Setelah ini, manfaatkan kode fungsi **sgd** yang telah dibuat pada percobaan SVM biner, lalu latih dengan memanggil fungsi **sgd** berulang kali sesuai banyaknya kelas yang ada pada data. Dengan demikian, proses *training* menghasilkan bobot sebanyak kelas yang ada pada *dataset*. Buatlah fungsi bernama *training* yang digunakan untuk melakukan proses *training* one-vs-rest.

```
1 def training(trainingset):
2     list_kelas = trainingset.keys()
3     w = {}
4     for kelas in list_kelas:
5         data_latih = trainingset[kelas]
6         label_latih = data_latih.pop(data_latih.columns[-1])
```



```
7     w[kelas] = sgd(data_latih,label_latih)
8     return w
9
```

Hasil dari proses *training* berupa *dictionary* dengan key berupa kelas dan value berupa bobot dari SVM yang mengenali kelas tersebut.

Lakukan proses *training* dengan memanggil fungsi **training** dan menempatkan hasilnya pada variabel **W**

```
W = training(trainingset)
```

Tampilkan isi variabel **W**

```
print(W)
```

3. Percobaan Proses Testing

Proses *testing* dilakukan dengan menghitung nilai *dot product* antara bobot hasil *training* dengan data uji. Kelas data ditentukan berdasarkan tanda (positif atau negatif) dari hasil *dot product* tersebut. Fungsi berikut mengimplementasikan proses *testing*.

```
1 def testing(W,data_uji):
2     prediksi = np.array([])
3     for i in range(data_uji.shape[0]):
4         y_prediksi = np.sign(np.dot(W,data_uji.to_numpy()[i]))
5         prediksi = np.append(prediksi,y_prediksi)
6     return prediksi
```

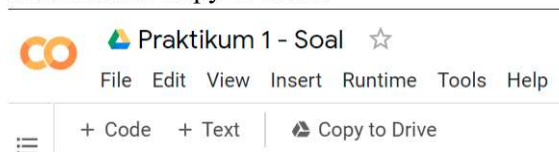
Lakukan pengujian menggunakan seluruh data uji. Bandingkan hasilnya dengan nilai label sebenarnya untuk menghitung berapa banyak data uji yang berhasil diprediksi dengan benar.

```
y_prediksi = testing(W,data_uji)
print(sum(y_prediksi==label_uji))
```

6.5 TUGAS

Pada tugas kali ini Anda mengimplementasikan metode *testing* pada metode one-vs-rest. Lengkapi kerangka *source code* pada *notebook* yang tersedia dan jawablah pertanyaan yang ada. Petunjuk pengerjaan soal:

1. Buka tautan berikut, pastikan Anda login menggunakan **akun student UB**.
<https://s.ub.ac.id/nbka106>
2. Klik tombol Copy to Drive



3. Beri nama file Praktikum 7 - Nama - NIM
4. Isilah *cell* yang kosong
5. Unduh file *.ipynb dengan cara klik **File -> Download .ipynb**
6. Kumpulkan file *.ipynb ke asisten

6.6 KESIMPULAN

Pada praktikum kali ini metode SVM telah dipraktikkan untuk melakukan klasifikasi terhadap dataset dengan 3 *class*. Pada dasarnya SVM hanya mampu mengklasifikasikan data dengan *class* biner (hanya 2

class), namun dengan strategi One-vs-rest atau One-vs-one, SVM dapat juga digunakan untuk mengklasifikasikan *dataset* dengan 3 *class*.

Setelah melakukan semua percobaan ini maka mahasiswa diwajibkan membuat kesimpulan.

Kesimpulan dari percobaan ini merupakan bagian penting untuk merangkum temuan utama dan relevansinya dengan tujuan percobaan. Mahasiswa harus mampu menyimpulkan:

- Apakah tujuan percobaan telah tercapai?
- Hubungan antara hasil percobaan dan dasar teori.
- Faktor-faktor yang memengaruhi hasil percobaan, termasuk kemungkinan kesalahan yang terjadi.

Kesimpulan harus disusun secara singkat, jelas, dan berdasarkan data yang diperoleh.